ROCKETMQ · LITETOPIC

LiteTopic 实践案例

轻量队列模型特性解析与场景落地

百炼平滑限流案例



CONTENTS

目录

01 **LiteTopic 特性**
轻量队列模型的四个核心特性详解

02 **百炼平滑限流案例**
从突发限流到通用限流插件

LITETOPIC · OVERVIEW

LiteTopic 特性

面向海量短生命周期会话的轻量队列模型，四个核心特性逐一拆解

01

PART 01 · 什么是 LITETOPIC

依然是发布 / 订阅模型

LiteTopic 与传统 Topic 本质相同——都是基于主题的 Pub/Sub 异步消息模型



共同本质

异步解耦 · 削峰填谷

基于主题的发布 / 订阅

Message → Topic → Consumer

区别在于：LiteTopic 在模型细节和使用场景上有自己的特点——更轻量的生命周期管理、更大规模的队列支撑、更灵活的订阅方式。接下来逐一展开。

PART 01 · 特性一

动态创建与销毁

不需要预配置，业务运行时数据面决定队列名字，随用随建、用完即走

传统 Topic

- 预先创建，生命周期长
- 与应用/业务生命周期一致
- 如交易系统中的 "Order" Topic
- 创建需审批、配置路由、分配资源

LiteTopic

- 动态创建和生成，运行时即建即用
- 业务数据面决定其名字
- 一个 SessionId、TaskId、RequestId 均可
- 粒度可粗可细，由业务场景决定

Session-A　Task-42　Request-xyz　Session-B　… 随时创建 …

PART 01 · 特性二

低成本 · 海量规模

从数千到千万级的跨越，TTL 回收策略保障资源不泄漏

传统 Topic 上限（商业实例）

3,000

受实例规格限制，创建成本高

LiteTopic 支撑规模

20W ~ KW

轻量化设计，创建与维护成本极低

TTL 自动回收

当某个 LiteTopic 不活跃时间超出配置后，自动销毁回收，释放 Quota 空间。无需人工清理，资源零泄漏。

PART 01 · 特性三

差异化订阅

打破 Group 集群消费的限制，每个客户端自由组织订阅关系

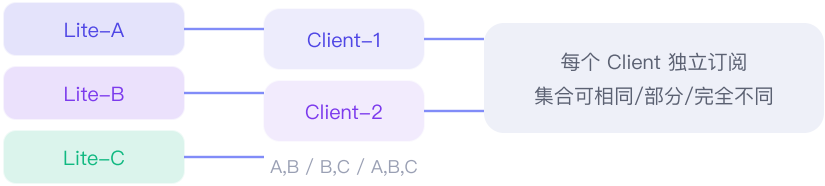
传统消费模型

基于 Group 分组的集群消费：相同订阅主题、相同处理逻辑的一组消费实例通过 Group 归组，一条消息在集群内被其中一个实例消费。



Lite 消费模型

单个客户端维度添加自己感兴趣的主题。多个客户端可订阅相同集合，也可部分相同、完全不同，灵活组织订阅关系。



PART 01 · 特性四

顺序消费

单个 LiteTopic 天然 FIFO，需要并发时可拆分队列

传统顺序消息

同一 Topic 下多个逻辑队列，ShardingKey（业务 ID）Hash 决定落入哪个队列。同一业务 ID 的消息一定在同一队列上保持顺序，但不同业务 ID 可能共享同一队列。



同 ID 必同队列（保序），不同 ID 可共享队列（交错排列）

LiteTopic 顺序

一个 LiteTopic 全局只有一个逻辑队列，默认就是 FIFO 消费。队列内全部是同一个 LiteTopic 的消息，天然有序，无需 ShardingKey。



CASE 1 · BAILIAN PLATFORM

百炼平滑限流案例

用 RocketMQ LiteTopic 重构大模型限流：从"直接拒绝"到"排队放行"

02

PART 02 · 2.1 背景

百万租户 × 稀缺 GPU，限流误差被成倍放大

1

百万级租户抢算力

百万级用户同时调用数十种大模型，GPU 是最稀缺的生产要素。

2

误判即双向损失

限流误判：用户超时、频繁重试；过载放行：算力被浪费、他人体验劣化。

3

误差被百万租户放大

一次误判的影响会沿租户规模扩散，小误差酿成大故障。

PART 02 · 2.2 百炼突发限流的特点

从"门口直接拦人"升级为"取号等候区"

传统限流（两态）

- 放行：在阈值内立即通过
- 拒绝：超出即返回 429
- 突发流量被直接误伤
- 只能"拒绝"，不能"缓冲"

平滑限流（三态）

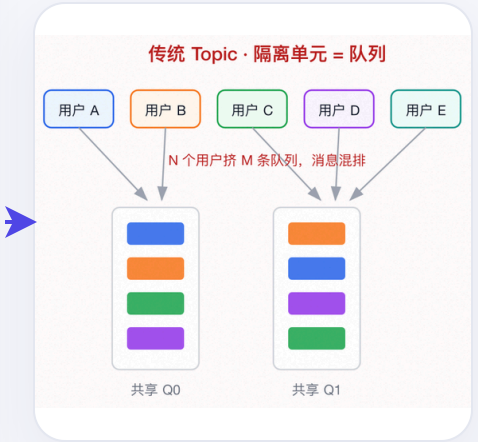
- PASS：能过就立即过
- QUEUE：过不了就排队等候
- REJECT：排满了才拒绝
- 用异步队列削峰填谷

排队等一等，好过直接被拒之门外

PART 02 · 2.3 基于 RocketMQ 的方案

限流识别 → MQ 缓冲入队 → 排队整形 → 模型服务





队头阻塞

传统消费模型自旋时HOL Blocking;
每个【用户+模型】需创建独立 `Topic/Group` 保证互不影响

运维成本高 · 无法合池

独立消费集群，无法跨用户/模型共享
资源利用率低

缺乏扩展性

特殊用户特殊配置，无法动态调整
新增用户或模型需新增配置，`topic`以`userId + model`命名

PART 02 · 2.4 落地效果

四个基于 LiteTopic 的适配

1 海量

为每个 userId、userId + model、APIkey 创建独立队列，精准限流。

2 动态

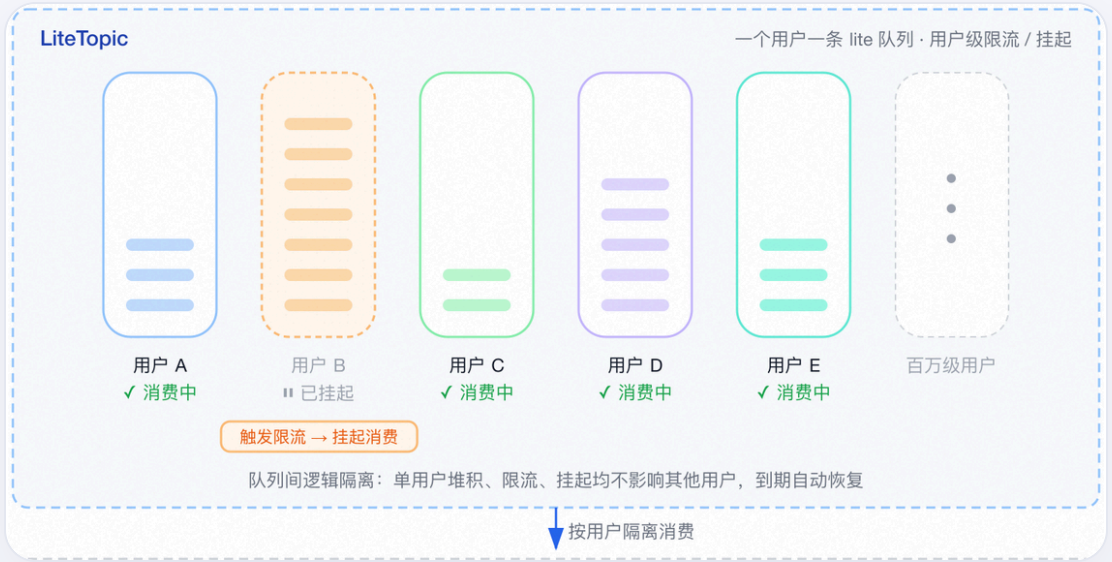
需要开启的用户，直接 Diamond 配置，入口处生效。

3 Suspend 语义

按队列挂起，毫秒到小时级控制。

4 精准订阅

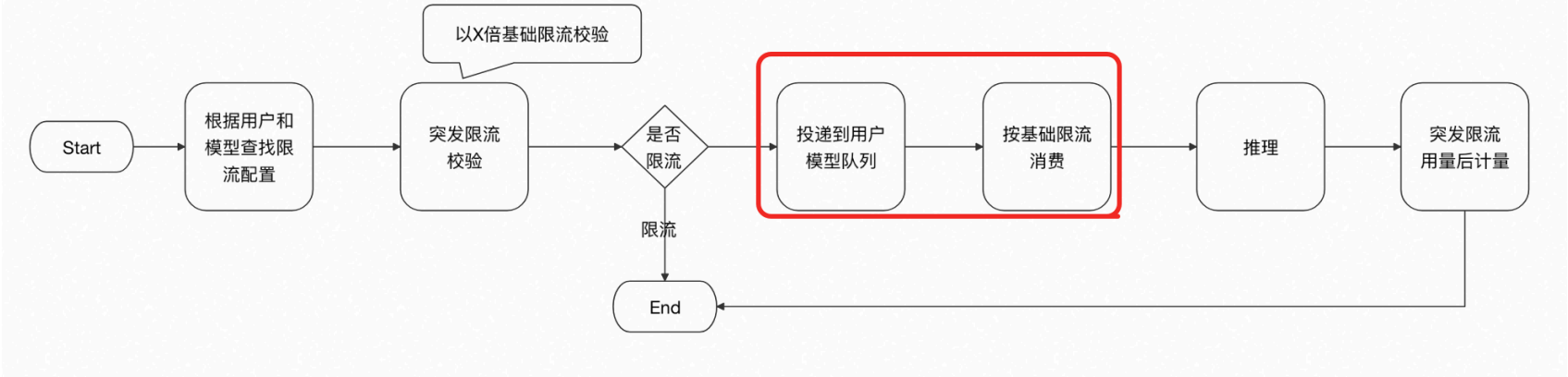
放行通知回到用户连接机器。



效果 10× 限流异常比例下降，用户可感知问题收敛，资源合池

PART 02 · 2.5 通用抽象

从百炼方案抽象出的四个关键点



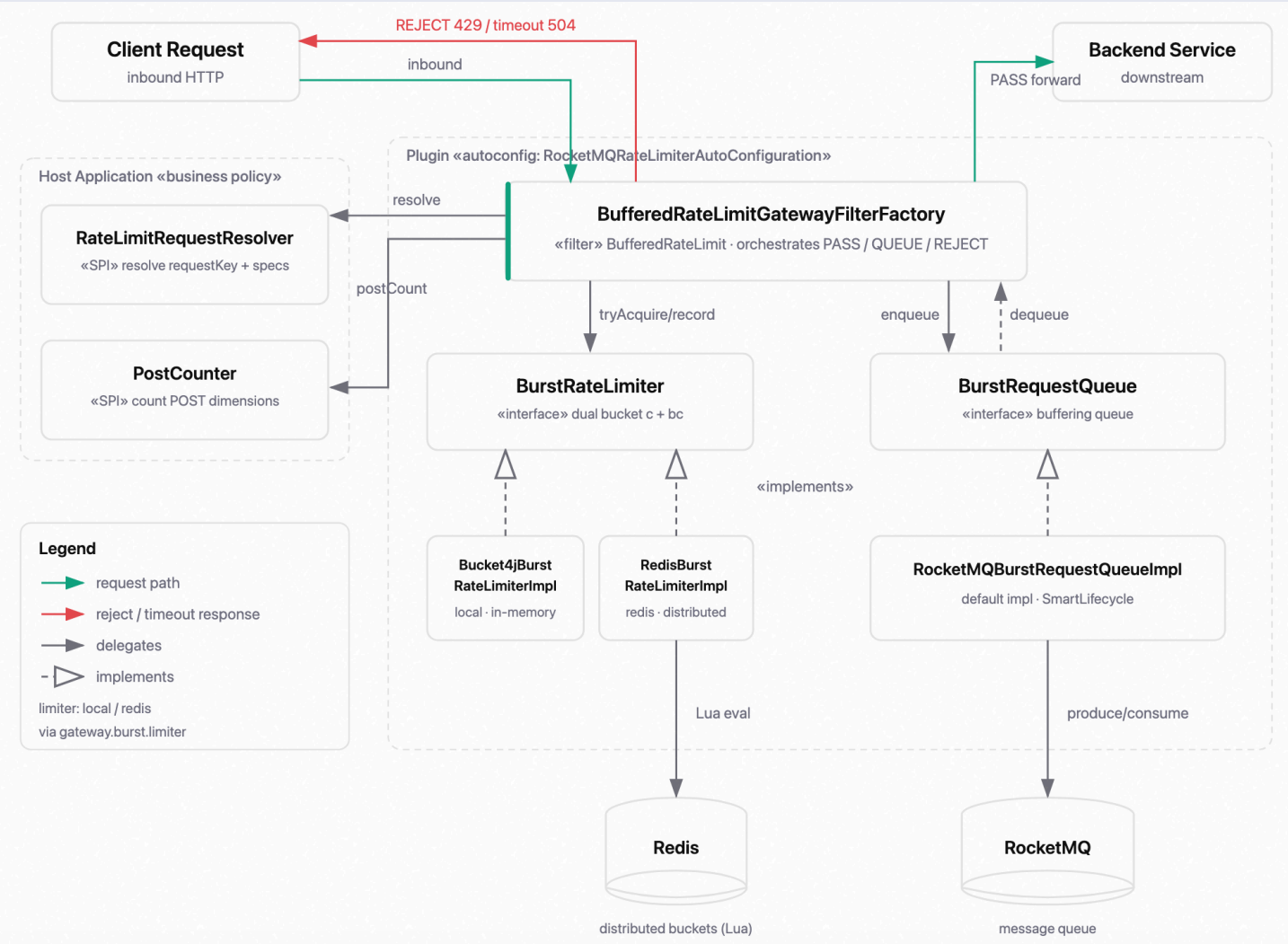
1 用户级限流配置
按用户 + 模型查找限流配置

2 突发限流识别
以 X 倍基础限流做突发校验

3 缓冲队列
超限请求投递到用户模型队列

4 放行及调用后处理
按基础限流消费，推理后计量

spring-cloud-gateway-rocketmq-ratelimiter



业务方只需实现 Resolver / PostCounter 两个 SPI，其余由插件自动装配

PART 02 · 2.7 关键能力

五项能力，开箱即用

1

削峰不阻塞

排队全异步，不拖垮网关事件循环

2

多维度同时限流

按用户、API、IP 等多维度原子管控

3

按结果计量

支持响应后按流量 / 行数计数

4

单机/分布式

内存版与 Redis 版可插拔切换

5

一键集成

Spring Boot Starter 交付

通用化的插件

THANK YOU

RocketMQ LiteTopic · 实践案例